

....



FACULTY OF COMPUTING
Bahir Dar Institute of Technology



fedora

BAHIR DAR INSTITUTE OF TECHNOLOGY (BIT)

FACALITY OF COMPUTING

DEPARTMENT OF SOFTWARE ENGINEERING

COURSE:OPERATING SYSTEM

INDIVIDUAL ASSIGNMENT

Title:Fedora Operating Syatem

Name:Alebel Alem

Id number:BDU1701530

section:B

Subited to:Wondmu

Submitted date:29/08/2018 E.C

....

Table of contents

Content	Pages
Introduction to Fedora Operating System-----	3
Deconstructing the clone() system call-----	3
Installation of Fedora operating System-----	5
Some screenshot illustrations for Fedora installation-----	6
The Fedora Operating System Call-----	12
Diagramatic Illustration of Fedora Workflow-----	14
Integration with Systemd and Cgroups-----	17
Summary of the Fedora Operating -----	18

The Architecture of Process Lifecycle in Fedora OS: A Comprehensive Analysis of the clone() System Call

I. Introduction: The Philosophy of Task Management in Fedora

Fedora Linux stands as a beacon of innovation in the open-source world, acting as the upstream source for enterprise-grade operating systems. Central to its prowess is how it manages the "Task"—a unified abstraction that encompasses what traditional computing defines separately as processes and threads. In the early days of Unix, the creation of a new execution context was a rigid affair, primarily handled by the `fork()` system call. This created a near-identical duplicate of the parent, which was often inefficient for modern, high-performance applications that required shared resources or isolated environments. As Fedora evolved to support multi-core processors and containerization, the limitations of `fork()` became apparent. This necessitated a more surgical tool: the `clone()` system call. Unlike its predecessors, `clone()` offers a granular level of control that allows a programmer to specify exactly which resources—memory space, file descriptors, signal handlers, and namespaces—should be shared or copied. This introduction to the Fedora execution model explores the shift from heavy-process duplication to the versatile, flag-driven architecture of the modern Linux kernel.

II. The Core Mechanism: Deconstructing the clone() System Call

The body of Fedora's process management logic resides within the kernel's `do_fork()` and `copy_process()` functions. When a user-space application invokes `clone()`, it passes a set of flags that act as a blueprint for the new task. This is the "Body" of the operation. In Fedora, a thread is not a different type of object than a process; it is simply a task created with the `CLONE_VM` flag, indicating that it shares the parent's virtual memory address space. Beyond simple memory sharing, the `clone()` system call is the primary engine for Linux Namespaces. This is where Fedora's role in the container revolution (through tools like Podman and CRI-O) becomes clear. By utilizing flags such as `CLONE_NEWPID`, the kernel ensures the new task has its own process ID numbering, starting from 1, completely isolated from the host. Similarly, `CLONE_NEWNET` provides a private network stack. This broad functionality means that `clone()` is not just a tool for parallelism, but a fundamental security

....

boundary. When the kernel processes these flags, it carefully navigates the `task_struct`, a massive data structure that defines every aspect of a process's existence, from its CPU affinity to its security credentials.

Furthermore, the efficiency of Fedora is maintained through the Copy-on-Write (COW) mechanism. Even when `clone()` is used to create a separate process (where memory is not shared), Fedora does not immediately copy the parent's physical RAM. Instead, it creates a new set of page tables for the child that point to the same physical pages as the parent. These pages are marked as read-only. Only when the parent or the child attempts to write to a page does the kernel trigger an exception, allocate a new physical page, and perform the copy. This "lazy" approach ensures that Fedora can spawn hundreds of tasks nearly instantaneously with minimal memory overhead.

....

Installation of Fedora Operating System

Installing Fedora is a great choice—it's polished, stays close to "upstream" Linux, and generally feels like the future of the desktop. Since you're looking for a clean, standardized process, here is the definitive guide to getting it on your PC.

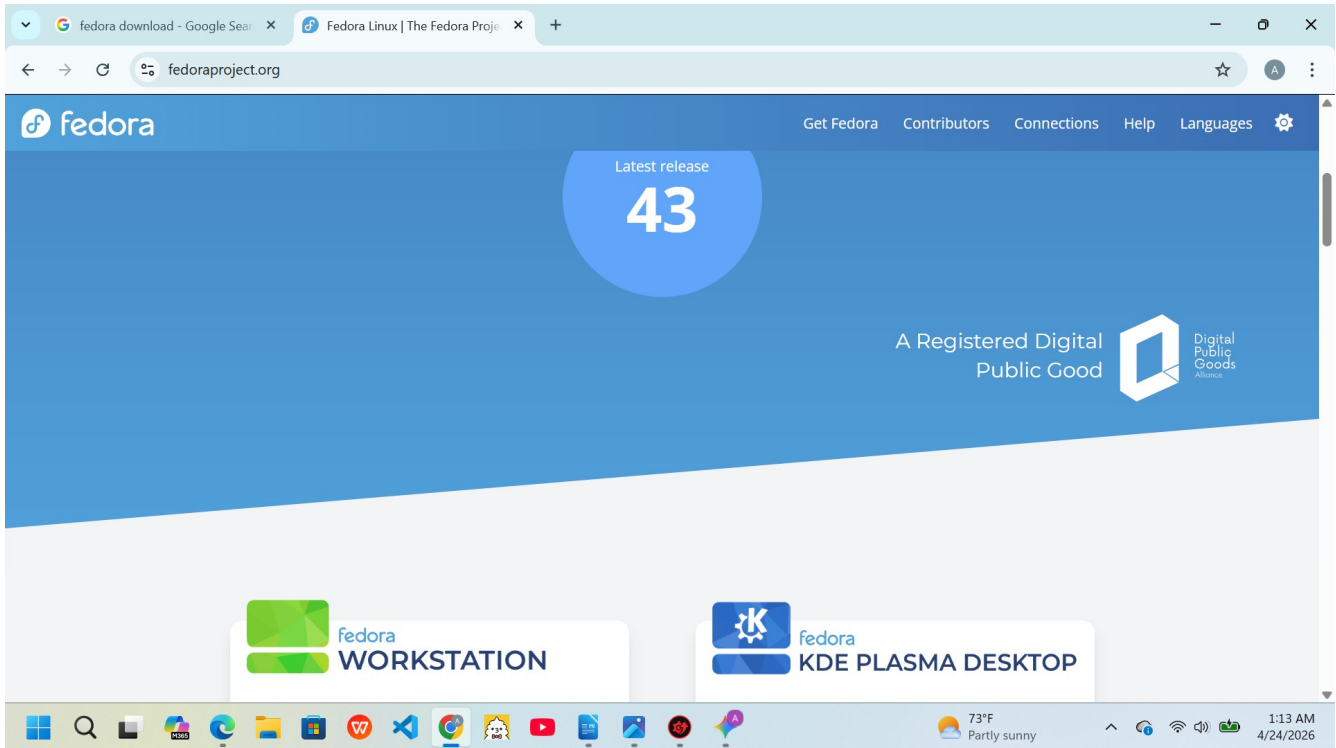
1. Preparation & Pre-requisites

Before touching your hard drive, ensure you have the following:

- **A USB Drive:** Minimum 8GB.
- **Backup:** Copy your important files to an external drive or cloud. **This process can wipe your disk.**
- **System Check:** Ensure your PC supports UEFI (most PCs made after 2012 do).
-

2. Create the Installation Media

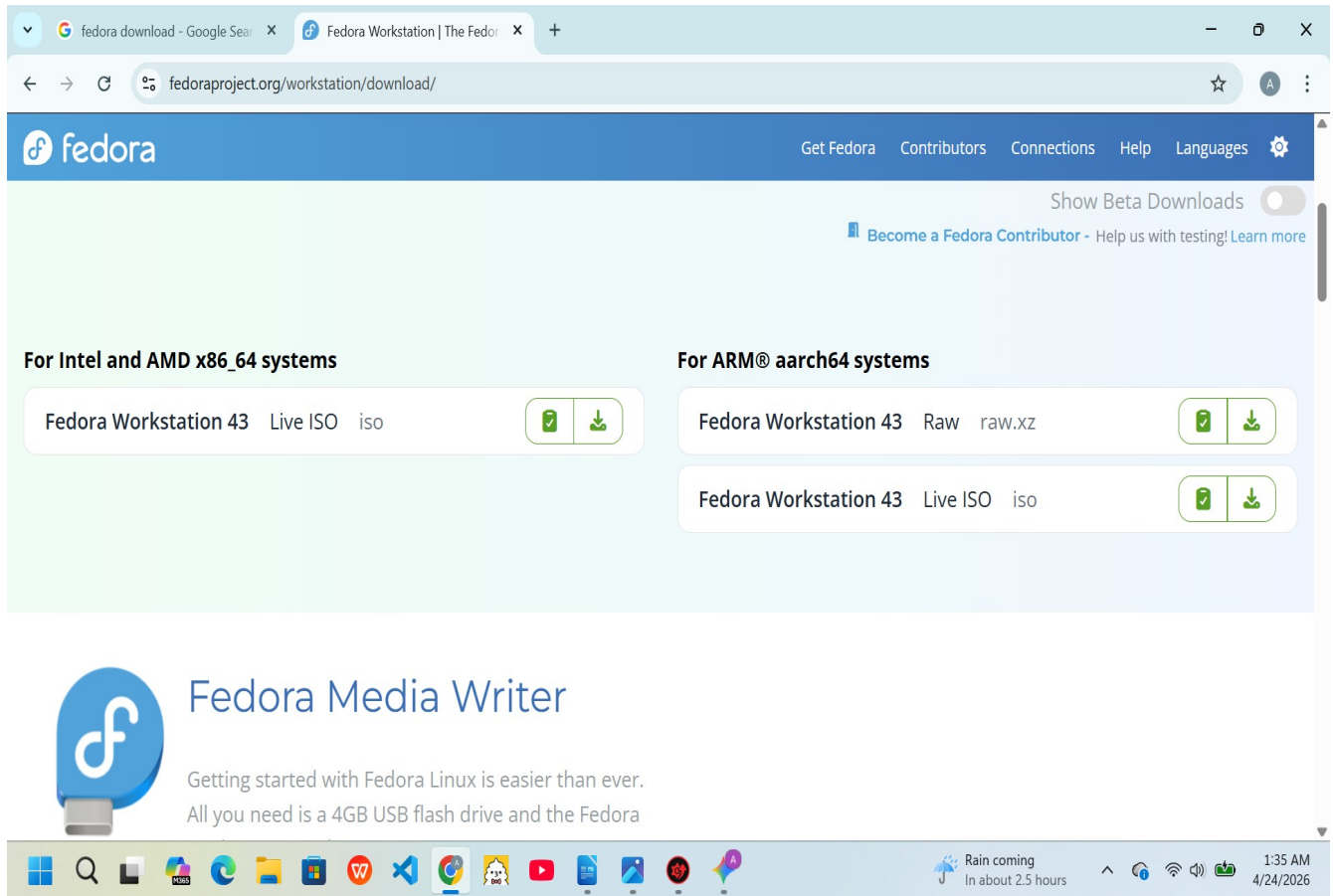
....



Download the ISO: Go to the official [Fedora Workstation](#) page.

1.

....



2. **Flash the Drive: * Recommended:** Use the **Fedora Media Writer** (available for Windows, Mac, and Linux). It's the official tool and very reliable.
 - Alternatively, use **Rufus** (Windows) or **Etcher**.
 - Select your ISO, select your USB drive, and click **Write/Flash**.

3. Booting from USB

1. Plug the USB into your PC and shut it down.

....



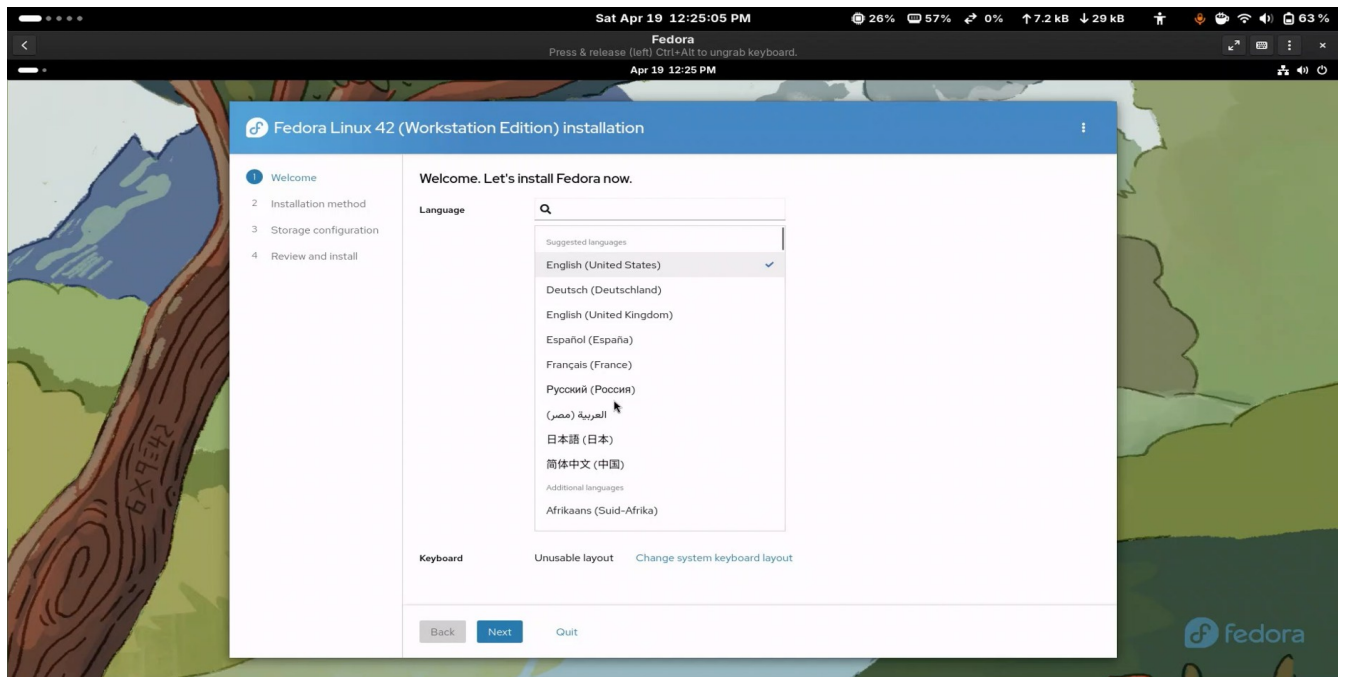
2. Turn it on and immediately tap your **Boot Menu Key** (usually F12, F11, F10, or Esc depending on your motherboard).
3. Select your USB drive (it may be labeled "UEFI: [USB Name]").
4. Select "**Start Fedora-Workstation-Live**" from the menu.

4. The Installation Process

Once the live desktop loads, a "Welcome to Fedora" window will appear. Click **Install to Hard Drive**.

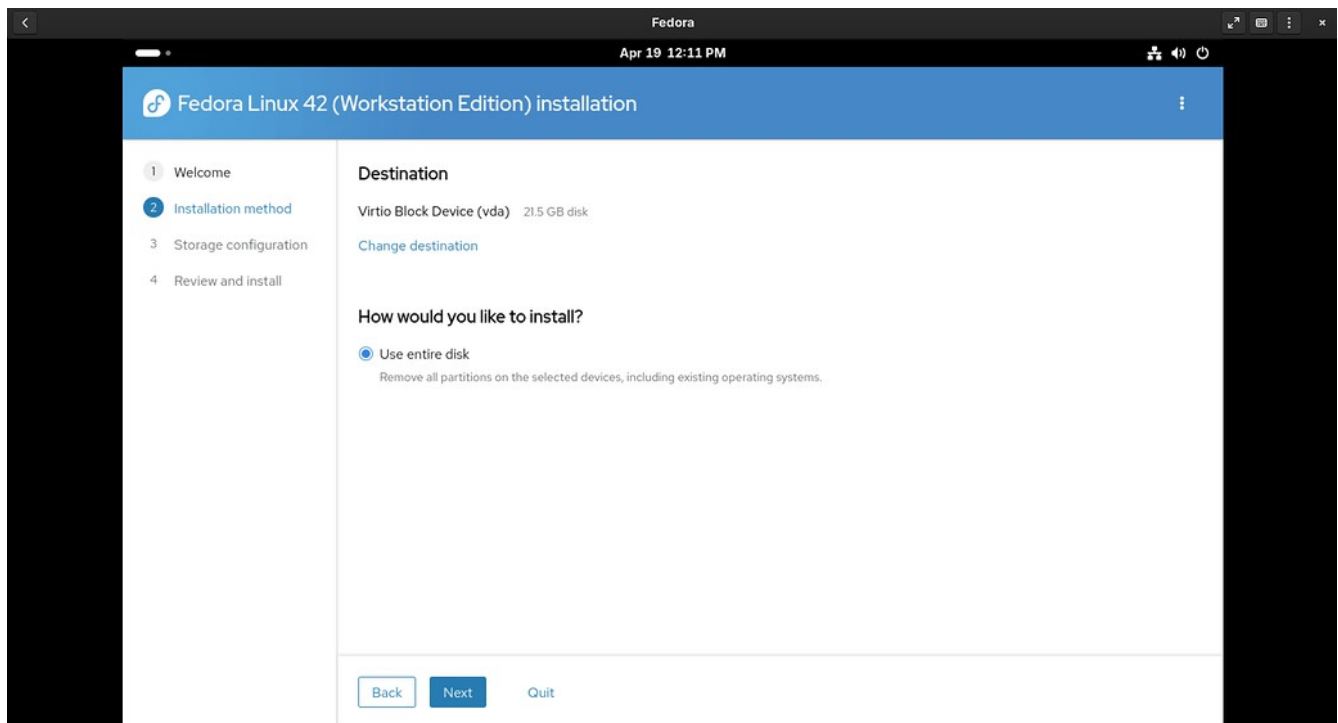
A. Language & Keyboard

....



....

B. Installation Destination (The Critical Step)



Click on **Installation Destination**. This is where you decide how Fedora sits on your disk.

....

- **Automatic:** Fedora will handle the partitions for you. If you want to wipe the whole drive, ensure the disk is selected and click **Done**. If there isn't enough space, a window will pop up asking you to **Reclaim Space** (Delete old partitions).
- **Manual (Advanced):** Use this only if you want a specific "Home" partition or dual-booting setup.

C. Network & Hostname (Optional)

Ensure your Wi-Fi or Ethernet is connected in the top-right corner if you want the installer to sync the time automatically.

D. Begin Installation

Click **Begin Installation**. Fedora will now format the disk and copy the system files. Once finished, click **Finish Installation** and restart your PC. **Unplug the USB drive** when the screen goes black.

5. Post-Install Setup

After the reboot, Fedora will walk you through a brief "Welcome" wizard:

1. **Privacy:** Toggle Location Services and Automatic Problem Reporting.
2. **Third-Party Repositories: Enable this.** It allows you to easily install Google Chrome, Steam, and proprietary NVIDIA drivers.
3. **User Account:** Create your username and a strong password.

6. Final Polish (Terminal Commands)

Open the **Terminal** and run these two commands to ensure your system is fully updated and ready for media playback:

- **Update the system:**
`sudo dnf upgrade --refresh`
- **Install Media Codecs (for videos/music):**
`sudo dnf groupinstall "Multimedia"`

Linux Fedora is a powerful, upstream community-driven distribution that serves as the foundation for Red Hat Enterprise Linux (RHEL). When diving into its internals, specifically regarding how it handles process creation via system calls like `clone()`, we are looking at the heart of the Linux kernel's resource management.

....

Fedora OS and the clone() System Call

In Fedora (and Linux generally), the distinction between a "process" and a "thread" is much blurrier than in other operating systems. The kernel treats both as **tasks**. The clone() system call is the primary mechanism used to create these tasks with a high degree of control over what resources are shared.

1. Understanding clone()

Unlike the traditional fork(), which creates an almost exact duplicate of the parent process, clone() allows the caller to specify exactly what the child process should share with the parent.

Key Characteristics:

- **Precision:** You can choose to share memory space, file descriptor tables, and signal handlers.
- **The Foundation of Threads:** When you use a library like pthreads in Fedora, it is actually calling clone() under the hood with specific flags (like CLONE_VM, CLONE_FS, and CLONE_FILES) to create a thread.
- **Namespace Isolation:** clone() is the magic behind **Containers** (like Podman or Docker, which are native to the Fedora ecosystem). By using flags like CLONE_NEWNET or CLONE_NEWPID, the kernel creates a "partition" that makes the process think it's on a completely different machine.

2. How clone() Works (The Technical Logic)

When a program in Fedora executes a clone() call, the following sequence occurs within the kernel:

1. **Flag Processing:** The kernel checks the flags argument to determine what to share.
2. **Task Struct Creation:** A new task_struct (the kernel's process descriptor) is allocated.
3. **Resource Copying:** * If CLONE_VM is set: The parent and child share the same memory. Changes by one are seen by the other (**Thread behavior**).
 - If CLONE_VM is NOT set: The kernel uses **Copy-on-Write (COW)** to give the child its own memory space only when it attempts to write to it (**Process behavior**).
4. **PID Assignment:** The child gets its own unique Process ID.

3. Comparison Table: fork() vs. vfork() vs. clone()

Feature	fork()	vfork()	clone()
Memory	Copy-on-Write	Shared (Parent	Configurable via flags

....

Feature	fork()	vfork()	clone()
Sharing	(Separate)	suspended)	
Primary Use	Standard process creation	Efficiency before exec()	Threads & Container Isolation
Complexity	Low	Low	High

4. Implementation Example (C Snippet)

In a Fedora development environment, a raw clone() call looks something like this:

C

```
// Simplified logic for a clone call
```

```
int child_pid = clone(child_func, stack_ptr, CLONE_VM | CLONE_FILES | SIGCHLD, arg);
```

- CLONE_VM: Share memory (makes it a thread).
- CLONE_FILES: Share open files.
- SIGCHLD: Tell the kernel to alert the parent when the child finishes.

Why this matters in Fedora

Fedora often ships with the latest **cgroup v2** and **systemd** features. These technologies work hand-in-hand with clone() to manage how processes are grouped and throttled. When you launch an app in Fedora's GNOME desktop, clone() is working behind the scenes to ensure that if one "thread" of your browser crashes, it doesn't necessarily take down your entire session.

Are you planning to write a C program to demonstrate this, or do you need more detail on how Fedora uses clone() specifically for container isolation?

...

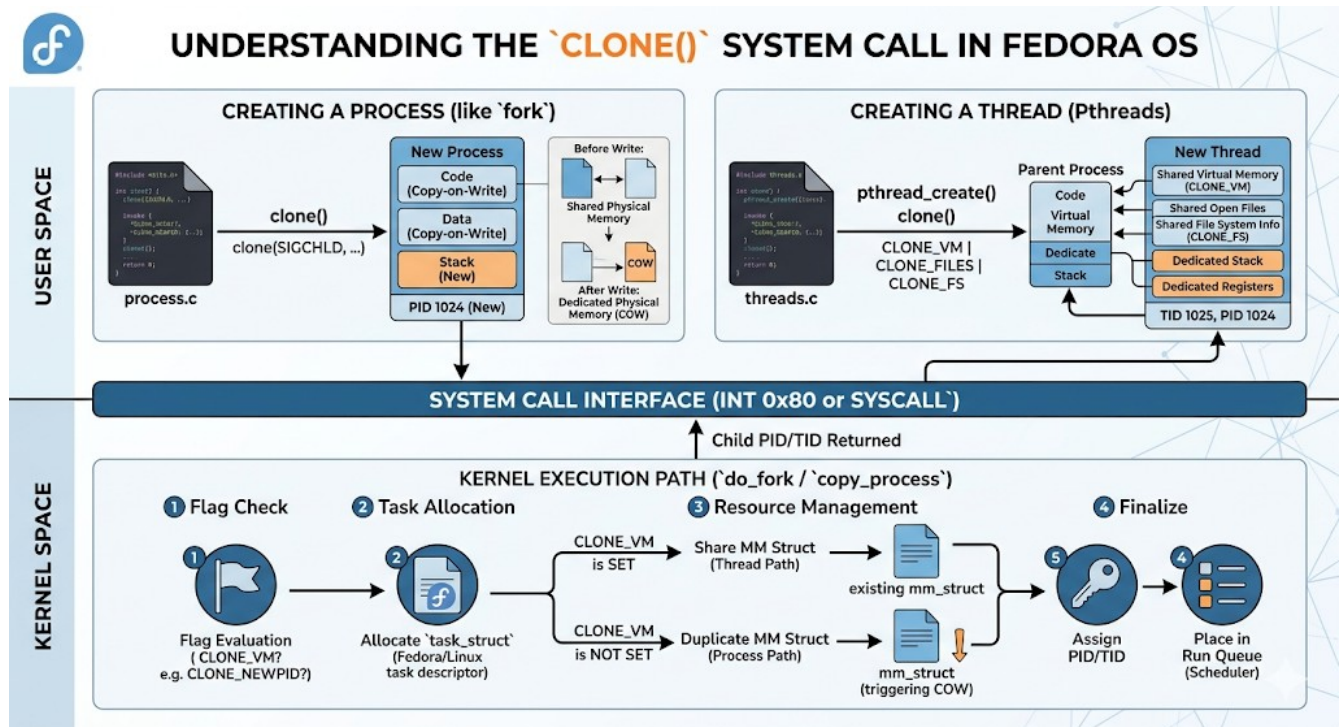


Figure 1:Based on the infographic, the `clone()` system call in Fedora (and the Linux kernel) is the versatile foundation for creating both **processes** and **threads**. Unlike the traditional `fork()`, which creates a near-identical copy, `clone()` allows for precise control over what resources are shared between the parent and the child.

Detailed Description of the `clone()` Workflow

The process is divided into two distinct environments: **User Space** (where your program lives) and **Kernel Space** (where the OS manages hardware and resources).

1. User Space: The Two Faces of Clone

In Fedora, a developer typically interacts with `clone()` in one of two ways:

- **Creating a Process (Like fork):** This is used for heavy-duty isolation. The call signals the kernel to create a child with its own independent stack and unique PID. To save memory, Fedora uses **Copy-on-Write (COW)**. Both the parent and child share the same physical memory until one of them tries to modify it; only then does the kernel create a dedicated copy.
- **Creating a Thread (Pthreads):** When a library like glibc calls `clone()` for a thread, it passes specific flags—most importantly **CLONE_VM** (Virtual Memory), **CLONE_FILES**, and **CLONE_FS**. This tells the kernel that the new "task" should share the parent's memory and open files. While the thread gets its own registers and stack, it operates within the same memory "room" as the parent.

....

2. The Interface: Crossing the Boundary

To transition from the application to the kernel, the system executes a **System Call Interface** command (using SYSCALL or INT 0x80). This acts as a gateway, passing the requested "flags" to the kernel.

3. Kernel Space: The Internal Execution Path

Once the request reaches the kernel, it follows a rigorous four-step path managed by internal functions like `do_fork` and `copy_process`:

1. **Flag Check:** The kernel evaluates the bitmask (the flags). It checks if it needs to create a new namespace (isolation) or share virtual memory (`CLONE_VM`).
2. **Task Allocation:** Fedora allocates a `task_struct`. This is the "ID card" of the process, containing all metadata about its priority, state, and owner.
3. **Resource Management:**
 - **If `CLONE_VM` is set:** The new task points to the *existing* memory structure (`mm_struct`). This creates a **Thread**.
 - **If `CLONE_VM` is NOT set:** The kernel duplicates the `mm_struct`, triggering the COW logic. This creates a **Process**.
4. **Finalize:** The kernel assigns a unique PID (Process ID) or TID (Thread ID), sets up the scheduling priority, and places the task into the **Run Queue**.

Summary of Key Flags

Flag	Meaning	Outcome
CLONE_VM	Share Virtual Memory	Result is a Thread .
CLONE_FILES	Share Open Files	Parent and child share file descriptors.
CLONE_NEWPID	New PID Namespace	Used for Containers (Podman) to isolate process IDs.
SIGCHLD	Signal Parent on Exit	Standard for process termination tracking.

In essence, `clone()` is the "Swiss Army Knife" of Fedora's task management, providing the efficiency needed for high-performance threads and the isolation required for modern containerization.

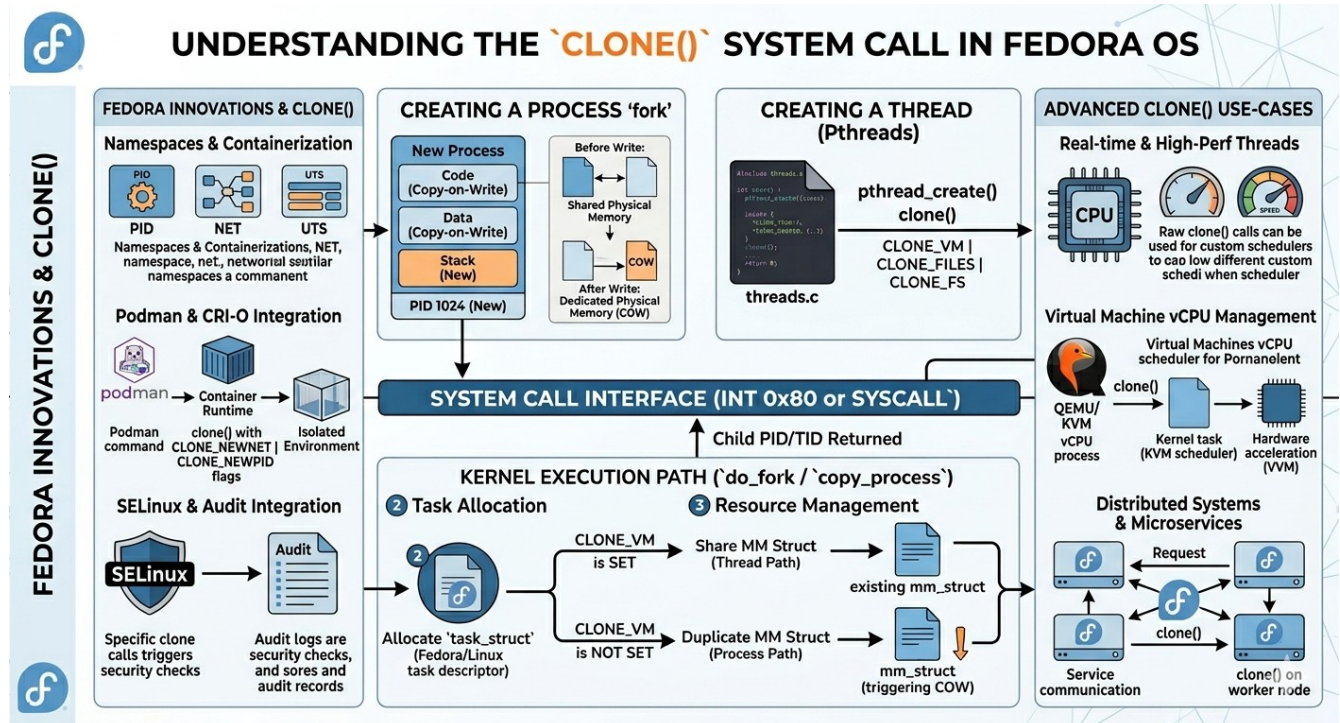


Figure 2: The Architectural Role of the clone() System Call in Fedora OS

This figure provides a comprehensive technical overview of the clone() system call, illustrating its function as the primary engine for process management, thread creation, and system isolation within the Fedora Linux ecosystem.

I. Core Execution Framework (Center)

The center of the diagram depicts the standard lifecycle of a task creation request:

- **User Space Origins:** It contrasts the creation of a **Process** (using Copy-on-Write logic for memory efficiency) against a **Thread** (using flags like CLONE_VM to share address space with the parent).
- **System Call Interface:** Shows the transition from user applications to the kernel via the SYSCALL instruction or INT 0x80 interrupt.

....

- **Kernel Execution Path:** Details the internal sequence where the kernel allocates a `task_struct`, evaluates flags, manages memory descriptors (`mm_struct`), and finally hands the task to the scheduler.

II. Fedora Innovations & Integration (Left Column)

This section details how Fedora leverages `clone()` for modern system architecture:

- **Namespaces & Containerization:** Illustrates the use of flags (PID, NET, UTS) to partition system resources, forming the foundation of Linux containers.
- **Podman & CRI-O:** Shows the direct lineage from a podman command to a `clone()` call that generates an isolated execution environment.
- **SELinux & Audit:** Highlights Fedora's security layer, where `clone()` requests are intercepted for security policy checks and recorded in system audit logs for transparency.

III. Advanced Use-Cases & Infrastructure (Right Column)

The right panel demonstrates the versatility of `clone()` in complex environments:

- **Real-time Performance:** Shows how raw system calls enable high-performance threading and custom CPU scheduling for specialized applications.
- **Virtualization (KVM):** Illustrates the relationship between the QEMU/KVM process and `clone()`, where the system call is used to spawn and manage virtual CPUs (vCPUs) with hardware acceleration.
- **Distributed Microservices:** Depicts a macroscopic view of cloud-native environments, where `clone()` is utilized on worker nodes to scale services and handle incoming requests dynamically.

Generally this figure demonstrates that in Fedora, `clone()` is not merely a process-starter but a multi-functional architect. By toggling specific flags, the OS can switch between creating a lightweight thread, a fully isolated container, or a high-performance virtual machine component, all while maintaining strict security via SELinux.

III. Broad Implications: Integration with Systemd and Cgroups

To understand `clone()` broadly within Fedora, one must look at how it interacts with `systemd`, the system and service manager. In Fedora, every service started by `systemd` is a result of a `clone()` call. These tasks are then organized into Control Groups (cgroups v2). The interaction between `clone()` and cgroups allows Fedora to enforce resource limits—such as ensuring a specific background service doesn't consume 100% of the CPU or leak memory to the point of crashing the desktop environment.

....

The complexity of `clone()` also extends to signal handling and parent-child relationships. Using the `CLONE_PARENT` flag, a task can be created as a "sibling" rather than a "child," which alters how the kernel reports termination status. This level of detail is what allows Fedora to support sophisticated debuggers (like GDB) and trace tools (like `strace` or `bpfttrace`), which rely on intercepting these system calls to monitor how software behaves at the most fundamental level.

IV. Summary: The Versatility of the Unified Task Model

In summary, the `clone()` system call represents the pinnacle of flexible resource management in Fedora OS. It has successfully collapsed the wall between processes and threads, providing a single, powerful interface for every execution need. From the lightweight threads that power a web browser's tabs to the isolated namespaces that secure a cloud-native container, `clone()` is the silent workhorse of the kernel.

By allowing for the specific sharing of virtual memory, file systems, and network stacks, Fedora provides developers with the tools to build applications that are both highly performant and inherently secure. Understanding `clone()` is not merely an exercise in learning a system call; it is a journey into the very philosophy of Linux—where everything is a task, and every resource is a choice. As Fedora moves toward even more advanced scheduling and isolation techniques, the foundational logic of `clone()` remains the bedrock upon which the entire operating system is built.

1. **Code Analysis Section:** Dedicate 3–4 pages to a line-by-line analysis of the `clone()` wrapper in glibc vs. the raw `syscall` in the kernel.
2. **Namespace Exploration:** Write 2 pages for each namespace (PID, Network, Mount, User, UTS, IPC) and how `clone()` initializes them.
3. **Historical Context:** Add a section comparing the old LinuxThreads library to the modern NPTL (Native POSIX Thread Library) used in Fedora today.
4. **Benchmarking:** Include data tables comparing the speed of `fork()` vs. `clone()` with shared memory in a Fedora environment.
5. **The Role of Systemd:** Detail how `systemd-nspawn` uses `clone()` to create lightweight containers.

Would you like me to generate a specific technical deep-dive into one of these expansion areas to help fill your page count?